ModalStripe: Modal Memory and Attention in Small Language Models

Atay Kim
Nazarbayev University
Astana, Kazakhstan
atay.kim@nu.edu.kz

November 5, 2026 (v0.5)

Abstract

We study whether a small language model benefits from combining modal memory with attention. ModalStripe replaces one of two global-attention blocks with a gated modal convolution followed by local attention. On two books excluded from training and primary model selection, it reaches character perplexity 5.592, compared with 6.047 for full attention and 5.964 for a local-plus-global control at the same training-character budget. The result persists with fixed feed-forward widths, but the advantage over attention falls to about 1.9% under a timing-calibrated training budget. CPU latency improves at long prefixes and the retained cache is roughly halved; most of that cache saving also occurs without the modal branch. A local-only baseline is a strong resource alternative, and prediction quality deteriorates beyond the trained context. The modal branch improves prediction at this scale; its training cost reduces the advantage under timing calibration.

1 Introduction

A hybrid language model can use less global attention without becoming a better model in practice. It may need more training, save cache while adding latency, or run quickly at a context length where its predictions have become worse. A useful comparison has to measure these effects together.

This paper examines one such design. ModalStripe has two blocks: the first combines a gated modal convolution with local attention, and the second retains full causal attention. The modal branch uses damped complex exponentials, allowing the same operator to run as an FFT convolution on complete sequences and as a compact recurrence during decoding. We compare it with two global-attention blocks and with a local-plus-global model that removes only the modal branch.

The experiments are deliberately small: width-64 character models, three optimization seeds, and transfer to two books. That scale makes a controlled audit practical, but limits what the results can establish. The main question is whether the modal branch earns its cost under explicit data, parameter and timing budgets. We find a consistent gain at equal training characters, a smaller gain after timing calibration, and meaningful weaknesses in context extrapolation. We also retain unfavorable results and distinguish the original comparisons from later exploratory checks.

2 Relation to existing models

ModalStripe uses established sequence-modeling ideas. Structured state-space models such as S4 provide recurrent and convolutional views of long-sequence operators [3]. H3 studies recall limitations and attention hybrids [4]; Hyena combines long convolutions with data-controlled gates [5]; and StripedHyena 2 develops convolutional multi-hybrids at scale [6]. These models provide the starting point for our study. We compare small, controlled implementations to understand what the modal branch contributes.

The connection to Fourier neural operators [2] is spectral computation, not demonstrated PDE-learning or resolution-transfer behavior. Zoology motivates separating language prediction from associative recall [7]; state-space duality and gated delta networks offer further approaches to efficient interaction and memory updates [8, 9]. Our character-perplexity results do not measure their capabilities or establish a new hybrid-model family.

3 Model and comparisons

All models start from random weights and share a fixed 128-symbol ASCII inventory, two blocks, width 64, four attention heads, sinusoidal positions [1], and an untied prediction head. A common causal stem adds a linear projection of the previous character embedding. Each mixer has a normalized residual update, followed by a normalized, two-layer GELU feed-forward network. There is no dropout.

The full-attention baseline uses global causal attention in both blocks. The local-plus-global control uses a window-8 local block followed by a global block. ModalStripe inserts the modal mixer before local attention in the first block. Both mixers act sequentially, sharing the block's first layer-normalization parameters. The global block is retained. Appendix A gives the full block definition and initialization.

For normalized input a_t, the modal branch writes $u_t = (W_v a_t) \odot \sigma(W_w a_t)$ and reads through $g_t = \sigma(W_r a_t)$. Its output is $W_o(z_t \odot g_t)$, where

$$z_t = \sum_{j=0}^{t} K_{t-j} \odot u_j, \qquad K_{\ell,k} = \operatorname{Re} \sum_{r=1}^{8} c_{r,k} p_r^{\ell}, \quad p_r = e^{-\alpha_r + \mathrm{i}\omega_r}, \quad \alpha_r > 0. \tag{1}$$

Poles are shared across channels; complex residues are channel-specific. The kernel uses nonnegative lags, so zero-padding to at least $2T - 1$ before FFT evaluation prevents circular wraparound. The equivalent decoding state is

$$s_{t,r,k} = p_r s_{t-1,r,k} + u_{t,k}, \qquad z_{t,k} = \operatorname{Re} \sum_{r} c_{r,k} s_{t,r,k}. \tag{2}$$

This finite state belongs to the modal branch. The complete model still has a growing global KV cache and quadratic full-sequence attention work.

We test two allocation rules. Approximate parameter matching gives about 121,000 parameters per model, but reduces the mixed block's feed-forward width from 261 to 124. A second condition fixes every feed-forward width at 256, allowing the hybrid to have more parameters. Parameter matching is tested at both training contexts; fixed width is tested at context 128. The comparisons separate a similar total budget from a fixed feed-forward allocation.

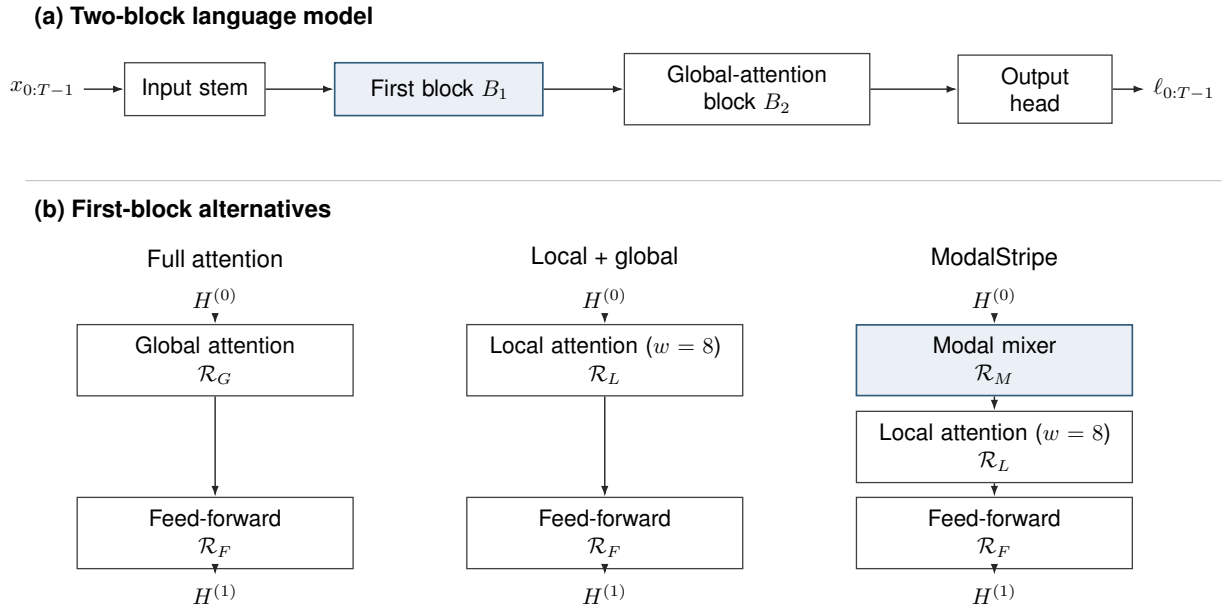


Figure 1: Model backbone and first-block alternatives. Each lower-panel box denotes a pre-normalized residual update. ModalStripe adds a modal update before local attention. All primary models retain the second global-attention block; weights are trained separately. Figure 4 expands the modal decoding operator.

4 Experimental setup

Data and training. Training uses the first 80% of normalized *Hamlet*, *Alice's Adventures in Wonderland*, and *Moby-Dick*; development uses the next 10%. Windows never cross document or split boundaries; starts are sampled uniformly from the pooled set of valid training starts. Gutenberg wrappers are stripped, typographic punctuation is normalized, and remaining non-ASCII characters are dropped. The fixed inventory avoids selecting characters from the test set. The source texts had already informed earlier exploratory work and are development sources at the project level.

The primary final books are *Frankenstein* and *The Adventures of Sherlock Holmes*, excluded from training and primary model selection. Primary model settings were fixed before final-book evaluation. We train for 1,800 updates with 2,048 input characters per update: batch 16 at context 128, or batch 4 at context 512. All runs use AdamW [10], weight decay 0.01, gradient clipping at 1, and cosine learning-rate decay to 10% of the initial rate. Seeds are 0, 1 and 2.

Each architecture/allocation/context condition receives the same seed-0, 400-update learning-rate sweep over $\{0.0005, 0.001, 0.002\}$. Selection uses equally weighted development-book loss in reset-128 windows. Every condition selects 0.002, the grid's upper endpoint. This is limited tuning, including for context-512 training, and neither optimal rates nor convergence are established.

Quality and training budgets. The primary score predicts every final-book character once, except the initial character, in consecutive reset-128 windows. We average loss equally across the

two books for each seed, then report the seed mean, sample SD, and its exponentiated mean as geometric perplexity. Seed SD is not a confidence interval across language distributions.

Two additional attention baselines test training cost. Literal elapsed-time matching gives attention each paired hybrid run's accumulated training-step time, with evaluation excluded. Because host load changed, we also use an interleaved calibration: the median of 60 paired hybrid/attention step-time ratios is 1.35459, giving attention $\lceil 1800 \times 1.35459 \rceil = 2439$ updates. This rule was specified before inspecting primary final scores and uses timing alone. It estimates a model-step time budget, not exact FLOPs or total runtime.

Measurement and audits. Measurements use an Apple M1 CPU, two PyTorch threads and float32; no GPU result is reported. Inference timing warms the models and randomizes model/seed order within nine repeats, measuring prefill with cache construction and a 32-character greedy continuation. Training-step timing uses three warmup updates and 20 interleaved repeats per seed. Tables average per-seed medians. Full attention uses a native library operator; local attention and modal operations are unfused reference implementations.

We check sampling, shifted targets, document boundaries, future-token influence, future gradients, cached/full agreement, local attention against a dense reference, and spectral finite-difference gradients. All 36 evaluated checkpoints pass the tested causality and cached-generation checks, including an 8,192-character prefix. Code, data, configurations, weights and raw timing records have saved hashes. Details and the later exploratory audits appear in the appendix.

5 Results

5.1 Prediction quality and training budget

Figure 2 shows a 7.5% perplexity reduction against full attention at equal training characters. Adding the modal branch improves on local plus global attention in all three seeds: paired loss differences are −0.0684, −0.0645 and −0.0602 nats. The direction persists with fixed feed-forward widths, where the hybrid has extra parameters (Appendix C). This supports adding the complete modal branch in this setup; it does not isolate Fourier modes from other possible extra gated mixers.

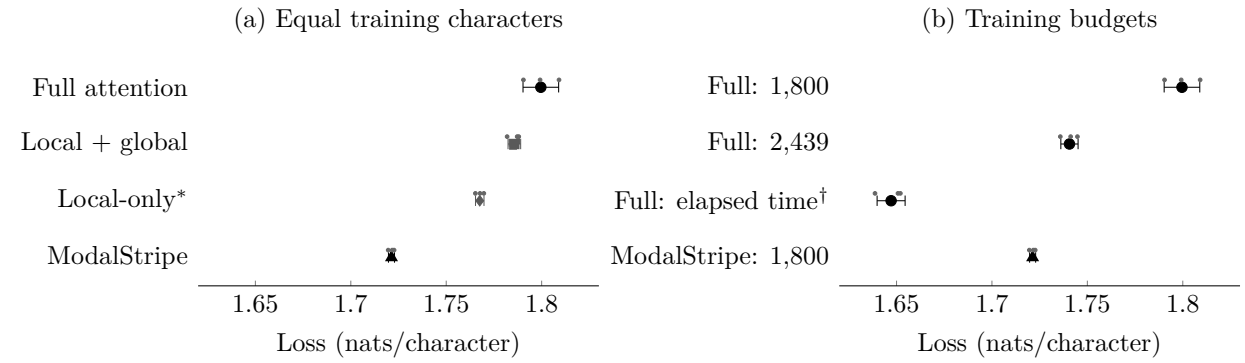


Figure 2: Final-book reset-128 loss for parameter-matched models; lower is better. Large markers are three-seed means, whiskers are sample SDs, and small markers are individual seeds. (a) All models receive 1,800 updates. *Exploratory local-only control. (b) Attention receives different training allowances. †Elapsed-time runs use 4,417, 4,609 and 4,146 updates; changing host load confounds this comparison. Exact scores are in Table 3.

Under the timing-calibrated allowance, attention improves to PPL 5.701, leaving a hybrid advantage of about 1.9%. Literal elapsed-time matching instead gives attention 4,417, 4,609 and 4,146 updates and reverses the result. Host-load drift prevents treating that reversal as clean compute matching, but it is important to retain. Neither comparison supports an unconditional training-efficiency claim.

5.2 CPU latency and cache

At short prefixes the hybrid is slower than full attention. At 8k, it has faster measured prefill and decoding and roughly half the retained cache (Figure 3). The local-plus-global control saves almost the same cache and runs faster. Removing one global-attention block accounts for most of the storage reduction; the modal state adds only 4 KiB.

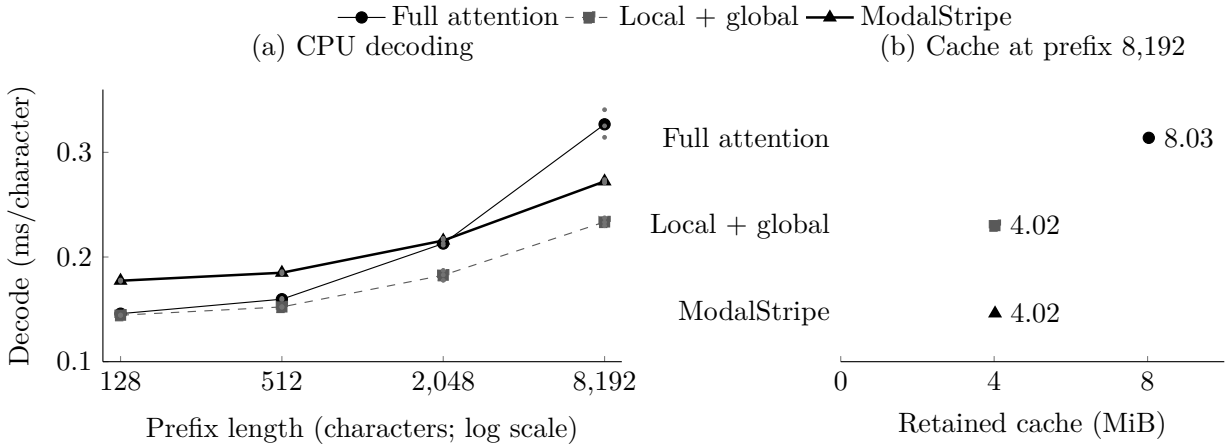


Figure 3: CPU decoding and retained cache for context-128-trained models; lower is better. Apple M1, float32, two threads, batch one, 32-character continuation. (a) Lines join means of three seed medians; small points are seed medians from nine interleaved repeats per seed. The horizontal axis is logarithmic. (b) Retained tensor payload at prefix 8,192, excluding weights and peak process memory. Table 4 gives exact values and prefill timing.

At training context 128, mean seed-median step times are 17.289 ms for attention and 24.411 ms for the hybrid. The ratio of those means differs from the median paired ratio used for calibration. At context 512 they are 26.445 and 27.903 ms. These measurements concern this implementation and hardware, not production kernels or GPU throughput.

A later local-only control reaches PPL 5.857. In its separate seed-0 timing audit, it retains only 8.25 KiB at 8k and decodes faster than the hybrid. It therefore offers a strong resource tradeoff for character prediction. Its bounded receptive field does not provide distant retrieval, however. The architecture control was added after primary outcomes were viewed and remains exploratory; its learning-rate choice still uses development data only.

5.3 Evaluation beyond the training context

Our first extrapolation test scored 128 continuation characters after a prefix. Even its 128-prefix condition therefore exceeded the positions seen in context-128 training. We retain it as a continuation test, not a fixed-window baseline. A corrective diagnostic predicts single targets from exactly

the named amount of history. It uses seed 0 and 64 fixed positions per book, shared across all input lengths.

Table 1: ModalStripe's corrective fixed-window diagnostic. PPL uses identical target positions at each history length. This small, single-seed audit was added after primary outcomes were inspected. Full comparisons are in Appendix C.

Training context	PPL at 128	PPL at 8,192
128	4.599	8.781
512	5.071	6.873

The drop in quality at longer histories remains clear in this diagnostic. Training at 512 reduces it but does not remove it. Longer history also changes absolute positional indices, so the test cannot separate positional extrapolation from the effects of additional context. Faster processing at 8k should not be read as preserved prediction quality there.

6 Discussion and conclusion

The modal branch improves short-window transfer prediction under both allocation rules. Its advantage over attention becomes small under the calibrated training allowance, while simpler attention controls expose useful resource alternatives. These results support a narrow small-model tradeoff.

The evidence covers two final books and three optimization seeds. Training losses are still improving late in the schedule, the selected learning rate is at the grid boundary, and the later context/local-only audits are exploratory. Larger token models, broader tasks, longer training, stronger mixer ablations and optimized GPU comparisons are needed for stronger claims. In particular, the hybrid retains one global-attention block: it has an $O(T)$ KV cache and quadratic attention work, not a bounded-memory or wholly subquadratic architecture.

Acknowledgments. OpenAI Codex assisted with implementation, auditing and manuscript preparation.

References

[1] A. Vaswani et al. *Attention Is All You Need*. 2017. arXiv:1706.03762.

[2] Z. Li et al. *Fourier Neural Operator for Parametric Partial Differential Equations*. 2020. arXiv:2010.08895.

[3] A. Gu, K. Goel, and C. Ré. *Efficiently Modeling Long Sequences with Structured State Spaces*. 2021. arXiv:2111.00396.

[4] D. Y. Fu, T. Dao, K. K. Saab, A. W. Thomas, A. Rudra, and C. Ré. *Hungry Hungry Hippos: Towards Language Modeling with State Space Models*. 2022. arXiv:2212.14052.

[5] M. Poli et al. *Hyena Hierarchy: Towards Larger Convolutional Language Models*. 2023. arXiv:2302.10866.

[6] J. Ku et al. *Systems and Algorithms for Convolutional Multi-Hybrid Language Models at Scale*. 2025. arXiv:2503.01868.

[7] S. Arora et al. *Zoology: Measuring and Improving Recall in Efficient Language Models*. 2023. arXiv:2312.04927.

[8] T. Dao and A. Gu. *Transformers are SSMs: Generalized Models and Efficient Algorithms Through Structured State Space Duality*. 2024. arXiv:2405.21060.

[9] S. Yang, J. Kautz, and A. Hatamizadeh. *Gated Delta Networks: Improving Mamba2 with Delta Rule*. 2024. arXiv:2412.06464.

[10] I. Loshchilov and F. Hutter. *Decoupled Weight Decay Regularization*. 2017. arXiv:1711.05101.

A Model and implementation details

The input at position t is $E[x_t]+W_{\text{prev}}E[x_{t-1}]+\text{PE}(t)$, with a zero previous embedding at a window boundary. The sinusoidal encoding uses base 10,000. Each mixer performs a pre-normalized residual update; the mixed block reuses its first normalization parameters before both sequential mixers. A second normalization precedes the feed-forward network, and a final normalization precedes the untied, bias-free output head. Linear maps are bias-free except the affine parameters of layer normalization. The network has no dropout.

In Eq. (1), $\alpha_r = \exp(\theta_r)$. Initial rates span $1/256$ to $1/2$ logarithmically; frequencies are equally spaced from 0 to π. Real and imaginary residue components are initialized with standard deviation $0.1/\sqrt{8}$. Other weights use the archived PyTorch defaults. Pole magnitudes below one do not by themselves bound the gain of the residual model.

For a sequence of length T, the kernel includes lags 0 through $T-1$. FFT evaluation pads to the next power of two at least $2T-1$, multiplies input/kernel transforms channelwise, inverts and retains the first T positions. This computes a one-sided convolution. Expanding Eq. (2) gives the same operator from zero initial state. Prefill constructs the recurrent state from the observed prefix. States are complex64 and other weights/activations are float32.

Approximate parameter matching chooses each block's feed-forward width as

$$d_{\text{ff}} = \max\left(8, \text{round}\,\frac{50{,}000 - P_M - 4d}{2d}\right),$$

where P_M counts mixer parameters and $4d$ counts the two layer normalizations. Full/local blocks use width 261, and the modal/local block 124. In the fixed-width condition, both blocks use 256. Nominal totals include an inherited, inactive 64-parameter frequency-feature vector and, in Modal-Stripe, one inactive frequency-gate scalar. No frequency features are supplied; these parameters receive no gradients. The paper preserves the archived totals rather than silently changing their definition.

AdamW uses moments $(0.9, 0.999)$ and epsilon 10^{-8}. The primary schedule is cosine with a 0.1 floor. Literal elapsed-time runs decay by elapsed budget; calibrated runs use a fixed 2,439-update schedule. In every condition, matching a seed matches the training-data stream; differently shaped weights need not share an initialization. Training at 512 uses fewer, longer examples at the same character budget and is tuned against reset-128 development loss. No independent convergence or long-window hyperparameter optimum is established.

A.1 Modal decoding operator and residual blocks

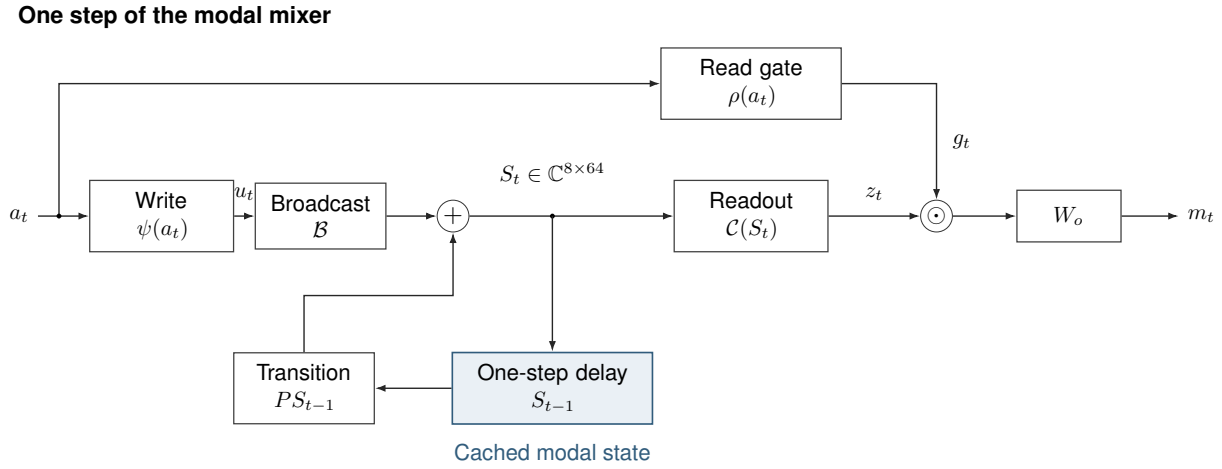
For $R = 8$ and $d = 64$, write the state as $S_t \in \mathbb{C}^{R\times d}$ and let $P = \text{diag}(p_1, \ldots, p_R)$. Define

$$u_t = \psi(a_t) = (W_v a_t) \odot \sigma(W_w a_t), \qquad g_t = \rho(a_t) = \sigma(W_r a_t), \tag{3}$$

$$\mathcal{B}(u_t) = \mathbf{1}_R u_t^{\mathsf{T}}, \qquad S_t = PS_{t-1} + \mathcal{B}(u_t), \quad S_{-1} = 0, \tag{4}$$

$$z_{t,k} = \mathcal{C}(S_t)_k = \text{Re}\sum_{r=1}^{R} c_{r,k} S_{t,r,k}, \qquad m_t = W_o(z_t \odot g_t). \tag{5}$$

Both state summands have shape $R \times d$. Residue multiplication uses no complex conjugation. P is shared across channels; the residues are channel-specific. Gates and projections are nonlinear or linear maps of the current representation as shown; the linear state transition does not make the complete network linear.



$\mathcal{B}(u_t) = \mathbf{1}_R u_t^{\mathsf{T}}$ copies each channel into eight modes before addition. $P = \mathrm{diag}(p_1, \ldots, p_8)$ acts on the mode axis. The blue buffer stores 4 KiB per example in complex64; the global attention cache is separate.

Figure 4: One modal decoding step. The write and read gates depend on normalized input. The fixed broadcast copies each write into all modes before the state addition. The delay is the modal state buffer; the full model also retains a global attention KV cache. Parameters are fixed during decoding.

Expanding the recurrence gives $S_{t,r,k} = \sum_{j=0}^{t} p_r^{t-j} u_{j,k}$, which recovers Eq. (1). Prefill constructs S_{T-1} from that weighted sum, separately from the FFT evaluation of sequence outputs. For $N = 2^{\lceil \log_2(2T-1) \rceil}$, the latter is

$$Z = \left[\mathrm{irFFT}_N \left(\mathrm{rFFT}_N(U) \odot \mathrm{rFFT}_N(K)\right)\right]_{0:T}. \tag{6}$$

The padding and slice compute the causal convolution without circular wraparound.

For the complete first block, the sequential updates are

$$H^m = H^{(0)} + \mathrm{Modal}(\mathrm{LN}_1(H^{(0)})), \tag{7}$$

$$H^a = H^m + \mathrm{LocalMHA}_8(\mathrm{LN}_1(H^m)), \tag{8}$$

$$H^{(1)} = H^a + \mathrm{FFN}_1(\mathrm{LN}_2(H^a)). \tag{9}$$

The two LN_1 applications share parameters and act on different inputs. The second block is

$$G = H^{(1)} + \mathrm{GlobalMHA}(\mathrm{LN}_3(H^{(1)})), \tag{10}$$

$$H^{(2)} = G + \mathrm{FFN}_2(\mathrm{LN}_4(G)), \tag{11}$$

where $\mathrm{FFN}_b(x) = W_{2,b}\,\mathrm{GELU}(W_{1,b}x)$.

For either attention operator, $q_t^{(h)} = W_Q^{(h)} a_t$, $k_t^{(h)} = W_K^{(h)} a_t$ and $v_t^{(h)} = W_V^{(h)} a_t \in \mathbb{R}^{16}$. With four heads,

$$A_{t,j}^{(h)} = \mathrm{softmax}_j \left(q_t^{(h)\mathsf{T}} k_j^{(h)} / \sqrt{16} + M_{t,j} \right), \tag{12}$$

$$b_t^{(h)} = \sum_j A_{t,j}^{(h)} v_j^{(h)}, \qquad \mathrm{MHA}(a)_t = W_A [b_t^{(1)}; \ldots; b_t^{(4)}]. \tag{13}$$

The mask is zero on $0 \le j \le t$ for global attention, or $\max(0, t-7) \le j \le t$ for local attention, and $-\infty$ otherwise. Attention selects among stored positions using a query. The modal branch instead reads an accumulated finite state; convolution–recurrence equivalence does not imply equivalence to arbitrary attention.

B Data, evaluation and evidence

Table 2: Normalized corpus sizes. PG is the Project Gutenberg ebook identifier. Final tests use each complete normalized book. The remaining 10% of each development-source text is unused in this study.

Work	PG	Characters	Train	Develop	Final test
Hamlet	1524	178,099	142,479	17,810	–
Alice	11	144,861	115,888	14,486	–
Moby-Dick	2701	1,220,651	976,520	122,065	–
Frankenstein	84	419,429	–	–	419,429
Sherlock Holmes	1661	562,332	–	–	562,332

The Gutenberg wrappers are removed; selected smart quotes, dashes, ellipses and nonbreaking spaces are mapped to ASCII equivalents, and remaining non-ASCII characters are discarded. The source works had informed the earlier project. The local protocol is not an independent preregistration. Primary model choices were fixed before the final books were fetched; the calibration rule was specified before primary final-score inspection. The local-only control and corrected fixed-window diagnostic are post hoc.

For book b and seed s, let $\ell_{b,s}$ be total next-character loss divided by the number of scored targets. Every target is counted once in reset-128 evaluation, so a book of N characters contributes $N-1$ targets. We form $m_s = (\ell_{1,s} + \ell_{2,s})/2$ and report its seed mean and sample SD; reported aggregate PPL is $\exp(\text{mean}_s\, m_s)$. Seed variability does not quantify uncertainty across books or tasks.

The original continuation audit uses eight blocks/book and scores 128 targets after each prefix. Scored input positions run from prefix−1 to prefix+126. The corrective audit uses 64 evenly spaced single-target positions/book, beginning no earlier than position 8,192, and exactly 128, 512, 2,048 or 8,192 preceding characters. This distinguishes the input window from a prefix followed by growing continuation history. The diagnostic remains sparse and single-seed, and it does not isolate positional from history-length effects.

Timing uses an Apple M1 with 8 GiB RAM, macOS 15.5, PyTorch 2.15.0, Python 3.13.3 and two threads. Inference records nine randomized/interleaved repetitions per model/seed after warmup, with 32 generated characters. Training timing records 20 repetitions after three warmup updates and excludes data sampling. Raw repetitions are retained. Optimized native full attention and unfused local/modal implementations are not equally engineered backends. The calibration median over 60 paired step-time ratios need not equal a ratio of displayed mean step times.

Pre-training checks cover the sampler's last valid window, target shifts, document boundaries, future influence and gradients, cache growth, local attention against a dense reference, and spectral finite differences. Trained-model checks cover all 36 evaluated checkpoints, including 8k-prefix cache agreement; maximum long-prefix logit discrepancy is 6.2×10^{-6}. An exact-block audit found no training substring match among 7,669 nonoverlapping 128-character final-book blocks. It does not guarantee semantic deduplication or general correctness.

Completed runs preserve full configurations, source copies, data hashes, checkpoints and raw measurements. JSON writes are atomic; incomplete runs restart. Run identifiers depend on configuration/code/data/environment records, and checkpoints are hash-verified before reuse. The companion experiment package runs through `experiments/rigorous/run_all.py`, archiving old and new evidence around regeneration. The code, saved checkpoints, raw results and manuscript are available at github.com/cerebrum101/modal-stripe.

The frozen core hashes are:

File	SHA-256
`study.py`	`77b87f56be8123b8d5e8250428b29b907a127b560bb07aab39f5c6caa1244540`
`model.py`	`afa4b39a20cac0503caa680ae5edcd56577813ba30794939ec0f5cc51e3b406d`

The source snapshots are Project Gutenberg ebooks 1524, 11, 2701, 84 and 1661. The manuscript evidence manifest identifies the result JSONs and all 36 evaluated run directories.

C Additional results

Table 3: Final-book reset-128 results. Models use approximately matched parameter counts and context-128 training. Loss is mean ± sample SD across three seeds. † marks the exploratory local-only control.

Model and budget	Updates	Test loss (nats)	PPL
Full attention	1,800	1.7996 ± 0.0093	6.047
Local + global	1,800	1.7857 ± 0.0033	5.964
ModalStripe	1,800	1.7213 ± 0.0016	5.592
Full, calibrated budget	2,439	1.7406 ± 0.0045	5.701
Full, elapsed-time budget	4,391 avg.	1.6472 ± 0.0073	5.192
Local-only†	1,800	1.7676 ± 0.0022	5.857

Table 4: CPU inference for context-128-trained models. Prefill includes cache construction. Cache counts retained tensors, including reserved capacity, not weights or peak process memory.

Prefix	Model	Prefill (ms)	Decode (ms/char)	Cache (KiB)
128	Full attention	0.661	0.1459	160.25
128	Local + global	0.742	0.1444	84.25
128	ModalStripe	1.005	0.1774	88.25
8,192	Full attention	201.524	0.3268	8,224.25
8,192	Local + global	115.441	0.2335	4,116.25
8,192	ModalStripe	130.785	0.2723	4,120.25

Table 5: Fixed feed-forward width 256 in each block, context-128 training and 1,800 updates. Unlike parameter matching, this condition permits additional parameters in ModalStripe. Loss is mean ± sample SD over three seeds.

Model	Parameters	Loss (nats)	PPL
Full attention	119,488	1.8015 ± 0.0084	6.058
Local + global	119,488	1.7843 ± 0.0032	5.955
ModalStripe	136,913	1.7060 ± 0.0036	5.507

Table 6: Per-book reset-128 PPL, geometric over three seeds. Calibrated attention uses 2,439 updates; other rows use 1,800. Local-only is exploratory.

Model	Frankenstein	Sherlock Holmes
Full attention	6.081	6.014
Local + global	5.995	5.933
ModalStripe	5.588	5.596
Full attention (calibrated)	5.724	5.678
Local-only	5.874	5.839

Table 7: Corrective exact-window diagnostic, seed 0 and the same 64 target characters/book. The input history is exactly the named length. This is a post hoc diagnostic, not a broad long-context benchmark.

Train context	Model	PPL, history 128	PPL, history 8,192
128	Full attention	5.734	11.048
128	Local + global	5.734	9.288
128	ModalStripe	4.599	8.781
512	Full attention	6.929	9.276
512	Local + global	5.292	8.983
512	ModalStripe	5.071	6.873
128	Local-only	5.240	11.852

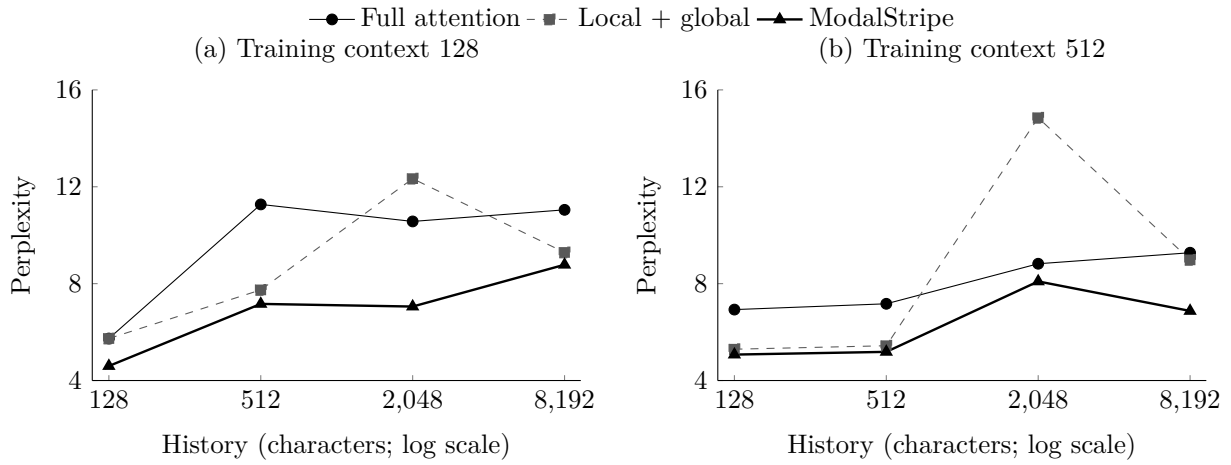


Figure 5: Corrective exact-history diagnostic for all three primary architectures, seed 0, using the same 64 target positions per final book. Lower perplexity is better. Horizontal axes are logarithmic. This exploratory test has no uncertainty estimate; absolute positional indices change with history length.

Table 8: Original prefix-plus-continuation test, three seeds and eight 128-character target blocks/book. Input positions extend beyond the prefix, including in the 128-prefix row.

Train context	Model	PPL, prefix 128	PPL, prefix 8,192
128	Full attention	12.077	14.983
128	Local + global	8.593	11.712
128	ModalStripe	7.315	9.100
512	Full attention	7.216	9.844
512	Local + global	5.932	10.244
512	ModalStripe	5.503	8.363

Table 9: Context-512-trained models tested with reset-128 windows on the complete final books. This evaluation context differs from training.

Model	Loss (nats)	PPL
Full attention	2.0051 ± 0.0060	7.427
Local + global	1.8211 ± 0.0115	6.179
ModalStripe	1.7508 ± 0.0028	5.759

Table 10: Development learning curves for approximately matched parameters and context-128 training. Values average three seeds and three source works. All checkpoints share the 1,800-update schedule; continued late improvement is not convergence.

Model	Update	Development loss (nats)
Full attention	300	2.1684
Full attention	600	2.0316
Full attention	1200	1.8878
Full attention	1800	1.8387
Local + global	300	2.0946
Local + global	600	1.9669
Local + global	1200	1.8575
Local + global	1800	1.8201
ModalStripe	300	2.0451
ModalStripe	600	1.9053
ModalStripe	1200	1.7942
ModalStripe	1800	1.7588

Table 11: Interleaved training-step timing: three warmup updates, then 20 repetitions per seed. Batch size is 16 at 128 and 4 at 512. Means of per-seed medians include forward, loss, backward, clipping and AdamW, but exclude data sampling.

Context	Model	Step (ms)
128	Full attention	17.289
128	Local + global	19.812
128	ModalStripe	24.411
512	Full attention	26.445
512	Local + global	23.439
512	ModalStripe	27.903

Table 12: Separate exploratory local-only timing at prefix 8,192, seed 0, nine randomized paired repeats. These measurements were collected separately from Table 4.

Model	Decode (ms/char)	Cache (KiB)
Full attention	0.3716	8,224.25
ModalStripe	0.3224	4,120.25
Local-only	0.1979	8.25

For batch one, two full-attention blocks retain $16d(T + 32) + 4d$ bytes of tensor payload under the allocation policy. ModalStripe retains

$$8d(T + 32) + 8wd + 8Rd + 4d,$$

corresponding to one global KV cache, local KV state, complex modal state and the previous embedding. At $d = 64, w = 8, R = 8$, the modal state is 4 KiB. Local plus global removes that state; two local blocks retain $16wd+4d = 8,448$ bytes. These formulas exclude weights, workspaces, allocator overhead and process memory. The local-only model's bounded receptive field does not establish distant recall; the project's earlier recall runs trained separate models and are not evidence of retrieval ability in these text-trained checkpoints.